

Computação Distribuída

Odorico Machado Mendizabal



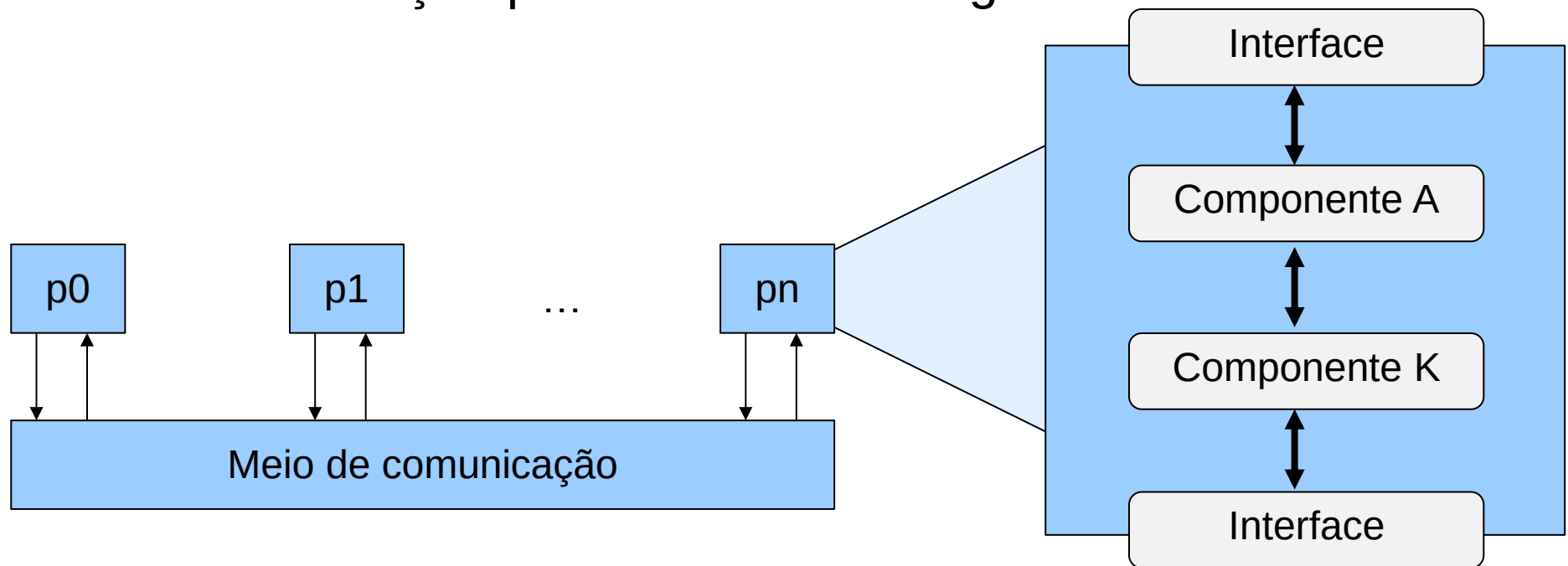
Universidade Federal de Santa Catarina – UFSC
Departamento de Informática e Estatística – INE



Modelos de Sistemas

Desenvolvimento de Sistemas Distribuídos

- Processos interconectados por uma rede de interconexão
 - Comunicação por troca de mensagens



Quais são as garantias ao projetista de algoritmos/sistemas distribuídos?

- Meio de comunicação é confiável?
- Aspectos do SO (segurança, desempenho, confiabilidade, ..)?
- Pilha de software (bibliotecas, dependências, interoperabilidade, ..)?

Intuição sobre o funcionamento de um sistema

- Observação experimental

- **Construir e observar.** Mesmo sem compreender porque o sistema funciona, esta experiência possibilita construir algo similar

- Modelagem e Análise

- **Simplificar e analisar.** Permite um entendimento mais profundo de partes do sistema, e possivelmente o modelo simplificado ainda corresponde à realidade

End-To-End Arguments in System Design

J. H. SALTZER, D. P. REED, and D. D. CLARK
Massachusetts Institute of Technology, Cambridge, Massachusetts 02139

This paper presents a design principle that helps guide placement of functions among the modules of a distributed computer system. The principle, called the end-to-end argument, suggests that functions be placed at low levels of a system may be redundant or of little value when compared with the cost of providing them at that low level. Examples discussed in the paper include bit-error recovery, security using encryption, duplicate message suppression, recovery from system crashes, and delivery acknowledgment. Low-level mechanisms to support these functions are justified only as performance improvements.

What Good are Models
and
What Models are Good?

Fred B. Schneider*
Department of Computer Science
Cornell University
Ithaca, New York 14853 U.S.A.

1. Refining Intuition

Refining Intuition

- **O que é um modelo?**

- Um conjunto de atributos e conjuntos de regras que regem como esses atributos interagem

- **Um modelo pode estar errado?**

"Uma teoria tem apenas a alternativa de estar correta ou errada. Um modelo tem uma terceira possibilidade: ele pode estar correto, mas ser irrelevante"

Manfred Eigen

Modelos de Sistema

- Desenvolvimento de algoritmos distribuídos parte da definição do modelo do sistema ao qual o algoritmo deve funcionar
 - Um modelo **descreve atributos** relacionados sincronia, comunicação, falhas, etc.
 - Modelo deve prover uma **visão simplificada** e ainda deve ser um **representação próxima da realidade**

Modelos com suposições fortes permitem soluções mais fáceis, mas possivelmente inadequadas para sistemas mais realistas. Por outro lado, suposições mais fracas exigem soluções mais fortes.

(tratabilidade do modelo versus precisão do modelo)



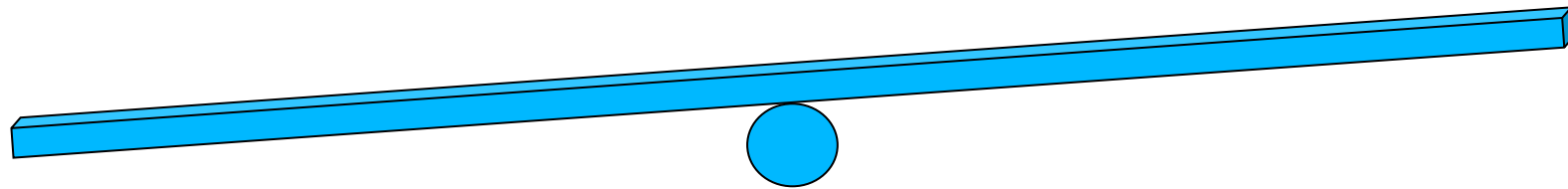
FB Schneider. What Good are Models and What Models are Good?
Distributed systems (2nd Ed.), 1993 - dl.acm.org

Bons modelos

- Modelos precisos
 - Representam a verdade sobre os objetos de estudo
- Modelos tratáveis
 - É possível analisa-los

Alta Precisão

Alta Tratabilidade



Baixa Precisão

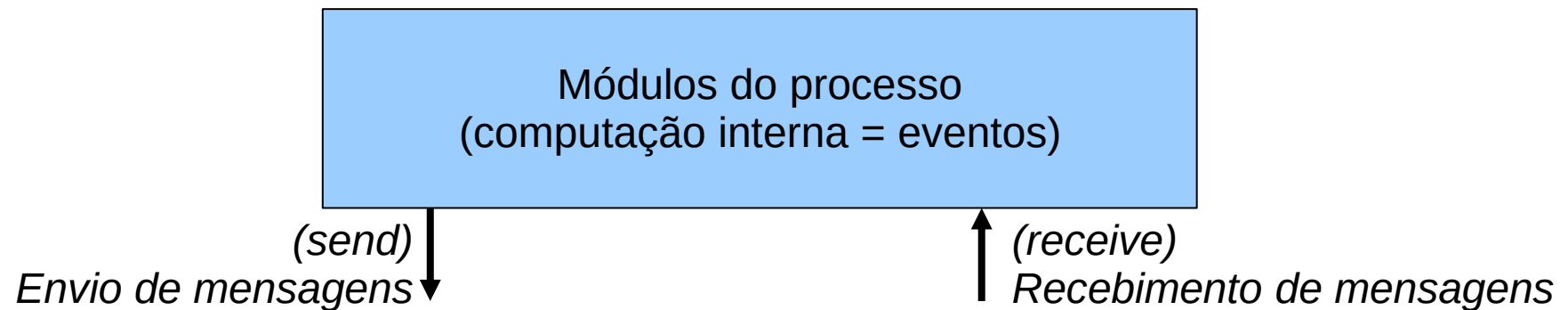
Baixa Tratabilidade

Bons modelos

- O que esperar de um modelo
 - Viabilidade (*feasibility*)
 - Quais classes de problemas podem ser resolvidos? (em um dado modelo)
 - Custo
 - Quão cara é a solução? (para um problema solúvel)

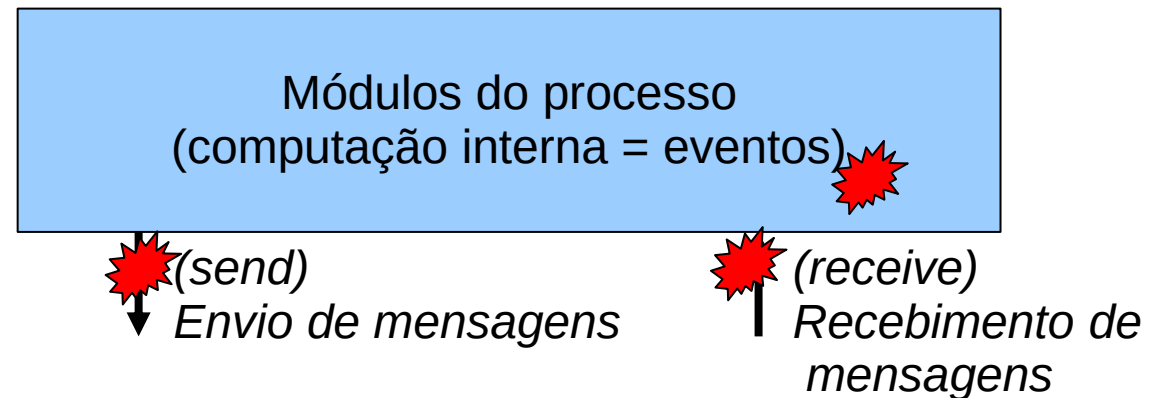
Modelos de Sistema – comportamento dos processos

- Suposições (por simplicidade)
 - Eventos do processo: local, *send* e *receive*
 - Existência de um relógio global (oráculo: sem acesso pelos processos)
 - Cada evento de um processo ocorre em um dado *tick* do relógio global

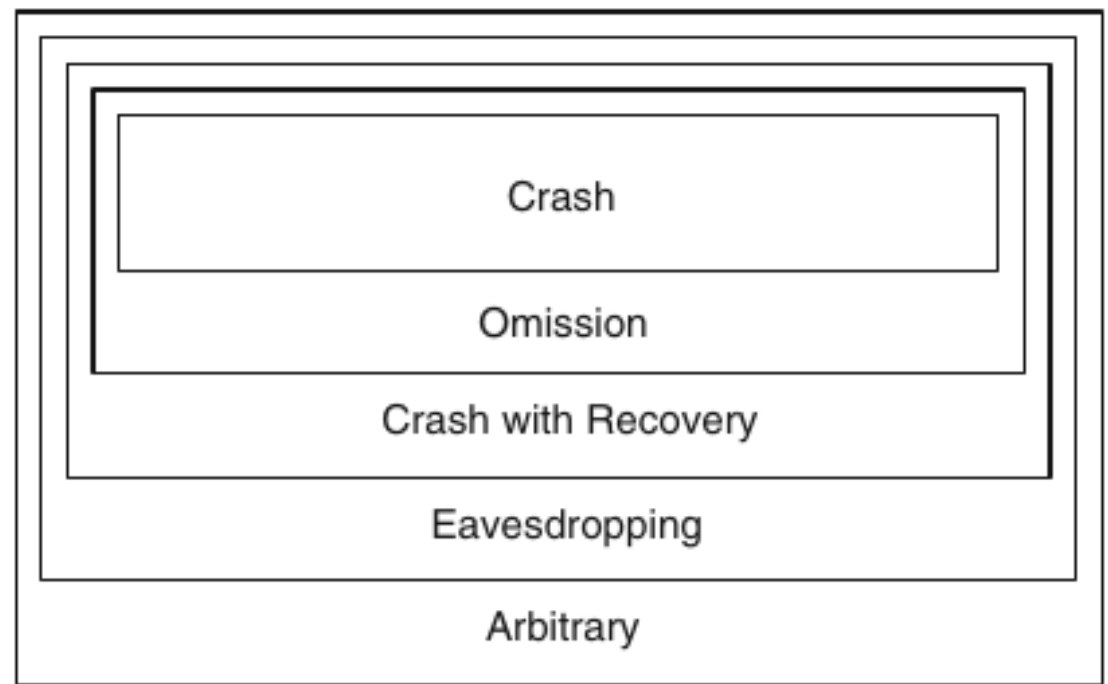


Modelos de Sistema – falhas dos processos

O comportamento falho é aquele que desvia do comportamento esperado



Falhas do processo



Fonte: C. Cachin, R. Guerraoui, L. Rodrigues. (2011)
Introduction to Reliable and Secure Distributed Programming.

Modelos de Sistema – falhas dos processos

- Colapso (*crash*):
 - O processo falha para de executar os seus passos. Executa corretamente até o instante t (após t para de executar eventos locais, *send* ou *receive*)
 - Após a falha, o processo nunca se recupera (não se torna correto novamente)
- Omissão (*omission*):
 - Uma falha de omissão ocorre quando um processo não envia ou recebe uma mensagem que supostamente deveria enviar ou receber
 - Estas falhas modelam comportamentos causados por *buffer overflow* ou congestionamento de rede, por exemplo

Modelos de Sistema – falhas dos processos

- Colapso com recuperação (*crash with recovery*):
 - O processo é *falho* se ele para e nunca recupera ou se ele fica indefinidamente parando e recuperando; caso contrário, ele é *correto*
 - Comportamento análogo à omissão, sendo que o processo para e não envia ou recebe mensagens. A diferença é que o processo sofre de amnésia (ele perde o estado local no momento da parada)
- Interceptação, “Escutar causalmente” (*eavesdropping*):
 - Um processo falho está exposto a um adversário. Mensagens trocadas (*send* ou *receive*) podem ser acessadas por um adversário

Modelos de Sistema – falhas dos processos

- *Arbitrária ou Bizantina (arbitrary ou Byzantine)*:
 - O processo desvia do comportamento esperado de qualquer forma. Ele pode representar parada, omissão, violação de passos determinísticos do algoritmo
 - Processos bizantinos podem atuar de forma coordenada (conluio) para comprometer o comportamento do sistema

Modelos de Sistema – suposições temporais

- **Sistemas assíncronos:**
 - Não há limites conhecidos para o tempo de execução de um processo ou atraso para entrega de mensagens
 - Suposição fraca: qualquer sistema é assíncrono
- **Sistemas síncronos:**
 - A velocidade relativa entre processos e a latência na entrega de mensagens é conhecida
 - Pode-se dizer que cada passo em um processo leva t unidades de tempo, onde $\text{lim_inf} < t < \text{lim_sup}$

Um algoritmo/protocolo desenvolvido para um sistema assíncrono também funciona em um sistema síncrono

Modelos de Sistema – suposições temporais

- **Sistemas com sincronia parcial:**
 - O sistema alterna entre comportamento síncrono e assíncrono
 - Há um instante em que, após um tempo, o sistema é síncrono por um período suficiente para permitir progresso

Qual modelo adotar e quando?

- **Perspectiva teórica**
 - Modelos ajudam a compreender como os atributos afetam a viabilidade e custo da solução
- **Perspectiva prática**
 - Como escolher um modelo?
 - O ambiente real (e.x., banco de dados, cluster de HPC, Sistema Operacional, etc.)
 - Risco de falhas (e.x., Bizantinas e segurança – falhas intencionais)

Boa intuição é indispensável!

Referências

Parte destes slides são baseadas em material de aula dos livros:

- *Coulouris, George; Dollimore, Jean; Kindberg, Tim; Blair, Gordon. Sistemas Distribuídos: Conceitos e Projetos. Bookman; 5ª edição. 2013. ISBN: 8582600534*
- *Tanenbaum, Andrew S.; Van Steen, Maarten. Sistemas Distribuídos: Princípios e Paradigmas. 2007. Pearson Universidades; 2ª edição. ISBN: 8576051427*
- *Cachin, Christian ; Guerraoui, Rachid; Rodrigues, Luís. Reliable and Secure Distributed Programming. 2ª Edição. Springer, 2011, XIX, 320 pages. ISBN-10: 3-642-15259-7. ISBN-13: 978-3-642-15259-7*

